

Python Lambda

[< Previous](#)[Next >](#)

A lambda function is a small anonymous function.

A lambda function can take any number of arguments, but can only have one expression.

Syntax

```
lambda arguments : expression
```

The expression is executed and the result is returned:

Example

[Get your own Python Server](#)

Add 10 to argument `a`, and return the result:

```
x = lambda a : a + 10
print(x(5))
```

[Try it Yourself »](#)

Lambda functions can take any number of arguments:

Example

Multiply argument **a** with argument **b** and return the result:

```
x = lambda a, b : a * b
print(x(5, 6))
```

[Try it Yourself »](#)

Example

Summarize argument **a**, **b**, and **c** and return the result:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

[Try it Yourself »](#)

ADVERTISEMENT



Tutorials ▼

References ▼

Exercises ▼

Certificates ▼

Search



In

HTML CSS JAVASCRIPT SQL PYTHON JAVA PHP HOW TO W3.CSS C C >

Python Numbers

Python Casting

Python Strings ▼

Python Booleans

Python Operators

Python Lists ▼

Python Tuples ▼

Python Sets ▼

Python Dictionaries ▼

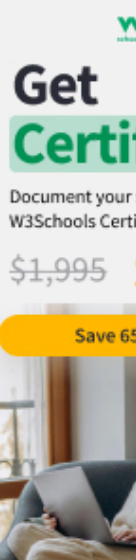
Python If...Else

Python Match

Why Use Lambda Functions?

The power of lambda is better shown when you use them as an anonymous function inside another function.

Say you have a function definition that takes one



COLOR PICKER



Python While Loops
Python For Loops
Python Functions
Python Decorators
Python Range
Python Lambda
Python Arrays
Python OOP
Python Classes/Objects
Python Inheritance
Python Iterators
Python Polymorphism
Python Scope
Python Modules
Python Dates
Python Math
Python JSON
Python RegEx
Python PIP
Python Try...Except
Python String Formatting
Python None

argument, and that argument will be multiplied with an unknown number:

```
def myfunc(n):  
    return lambda a : a * n
```

Use that function definition to make a function that always doubles the number you send in:

Example

```
def myfunc(n):  
    return lambda a : a * n  
  
mydoubler = myfunc(2)  
  
print(mydoubler(11))
```

[Try it Yourself »](#)

Or, use the same function definition to make a function that always *triples* the number you send in:

Example

```
def myfunc(n):  
    return lambda a : a * n  
  
mytripler = myfunc(3)  
  
print(mytripler(11))
```

[Try it Yourself »](#)

Or, use the same function definition to make both functions, in the same program:

Example

```
def myfunc(n):  
    return lambda a : a * n  
  
mydoubler = myfunc(2)  
mytripler = myfunc(3)  
  
print(mydoubler(11))  
print(mytripler(11))
```

Try it Yourself »

Use lambda functions when an anonymous function is required for a short period of time.

Exercise [?]

What will be the result of the following code:

```
x = lambda a, b : a - b  
print(x(5, 3))
```

- ☐ 15
- ☐ 8
- ☐ 3
- ☐ 2

Submit Answer »